



Tech Stack

◆ Frontend

- **Framework:** Next.js (React with SSR/CSR)
- **Styling:** Tailwind CSS

◆ Backend

- **Framework:** Node.js + Express
- **Database:** PostgreSQL
- **ORM:** Prisma / Any of your choice

◆ Authentication

- NextAuth.js (*or Clerk*)

◆ File Uploads (Optional)

- Cloudinary / UploadThing / supabase / firebase

◆ Deployment

- Frontend: Vercel
 - Backend: Vercel / Render
 - Database: Railway / Supabase
-



System Architecture

Frontend (Next.js)



API Layer (Next.js API / Express)



Database (PostgreSQL via Prisma)

Database Schema (Prisma)

```
model User {
  id      String  @id @default(cuid())
  email   String  @unique
  username String  @unique
  posts   Post[]
  comments Comment[]
  votes   Vote[]
}

model Community {
  id      String  @id @default(cuid())
  name    String  @unique
  slug    String  @unique
  posts   Post[]
  createdAt DateTime @default(now())
}

model Post {
  id      String  @id @default(cuid())
  title   String
  content String
  imageUrl String?
  community Community @relation(fields: [communityId], references: [id])
  communityId String
  author   User    @relation(fields: [authorId], references: [id])
  authorId String
  comments Comment[]
  votes    Vote[]
  createdAt DateTime @default(now())
}

model Comment {
  id      String  @id @default(cuid())
  content String
  post    Post    @relation(fields: [postId], references: [id])
  postId  String
  author   User    @relation(fields: [authorId], references: [id])
  authorId String
  createdAt DateTime @default(now())
}
```

```
model Vote {  
  id    String @id @default(cuid())  
  type  String  
  user  User   @relation(fields: [userId], references: [id])  
  userId String  
  post  Post   @relation(fields: [postId], references: [id])  
  postId String  
}
```
